

R4Tun: LLM-driven adaptive segmental tunnel lining segmentation in point clouds

Authors: Xinghui Tao, Guangming Wang *, Jelena Ninić, Brian Sheil

Affiliations: a: Construction Engineering, University of Cambridge, Trumpington Street, Cambridge, CB2 1PZ, Cambridge, UK b: Department of Engineering, Durham University, Stockton Road, Durham, DH1 3LE, Durham, UK

Corresponding author: Guangming Wang, gw462@cam.ac.uk, Construction Engineering, University of Cambridge, Trumpington Street, Cambridge, CB2 1PZ, Cambridge, UK

Abstract

Automated inspection of segmental tunnel linings is increasingly required to support structural health assessment and long-term operational safety, given the scale, frequency, and access constraints of modern tunnel networks. While modern laser scanning can capture large, geometrically rich point-cloud datasets, reliably segmenting these into individual components remains a challenging step for downstream deformation analysis and defect localisation. Current pipelines depend heavily on expert-driven hard-coded features, annotated datasets, or specifically tuned parameters, undermining their adaptability for field deployment at scale. This paper presents R4Tun, a large language model (LLM) driven adaptation framework that augments an expert-designed tunnel segmentation pipeline with bounded, context-aware parameter tuning. R4Tun uses structured context comprising memory, state, and knowledge to adjust stage parameters under changing tunnel conditions without expert tuning. The framework was evaluated on 30 Seg2Tunnel subsets spanning 13 regular and 17 complex tunnels using a stepwise ablation of memory, state, and knowledge across three LLMs (Claude Opus 4.6, GPT-5.4, Gemini 3 Flash). Depending on the LLM, the full design improved mean mIoU by 108.7–118.7% overall (from 0.150 to 0.313–0.328), by 74.6–83.8% for regular tunnels (from 0.291 to 0.495–0.535), and by 302.4–321.4% for complex tunnels (from 0.042 to 0.169–0.177), with state contributing the largest incremental gain. Cross-model analysis of 1,350 adapted parameter files revealed that all three LLMs independently converge on the same critical parameters and tunnel-family clustering, suggesting that adaptations are driven by objective tunnel characteristics rather than LLM-specific biases. The results indicate that R4Tun improves adaptability across varying tunnel conditions by adapting stage parameters from reference parameter values, supporting more reliable automated tunnel lining inspection without expert tuning.

Keywords: Segmental tunnel lining; Point cloud segmentation; Tunnel inspection; Large language models; Parameter adaptation; Multi-agent systems

Highlights

- Expert-tuned tunnel point-cloud segmentation degrades when tunnel conditions vary from the reference configuration.

- R4Tun adds an LLM-driven adaptation layer that adjusts pipeline parameters using structured context from memory, state, and domain knowledge.
 - The framework was validated on 30 Seg2Tunnel subsets covering two tunnel diameters, two ring lengths, two segment counts, and two joint types across three LLMs.
 - The full design improved mean mIoU by more than 100% overall, with the largest relative gains on complex tunnels (+302–321%).
 - Analysis of 1,350 adapted parameter files shows that three independent LLMs converge on the same critical parameters and adaptation patterns.
-

1. Introduction

1.1 Construction and infrastructure context

Automated inspection of segmental tunnel linings is increasingly required to support structural health assessment and long-term operational safety, given the scale, frequency, and access constraints of modern tunnel networks (Attard et al., 2018). Unlike 2D image data, which collapses geometry and is highly sensitive to viewpoint, lighting, and surface appearance, 3D spatial data preserves true geometric relationships and scale, making it better suited to characterising tunnel lining shape, alignment, and deformation in complex underground environments. Reliable segmentation of structural components from these data is therefore a prerequisite for downstream tasks such as deformation assessment and defect localisation (Weidner and Walton, 2024). Although modern laser-scanning technologies enable high-fidelity tunnel capture, the resulting measurements typically contain mixed structural elements, occlusions, and noise, making direct extraction of lining components challenging (Huang et al., 2021; Sjölander et al., 2023). In practice, engineers must operate across projects where tunnel geometry, lining condition, and data acquisition conditions vary substantially. As a result, expert oversight remains necessary to establish trust in automation (Montero et al., 2015; Strauss et al., 2020; Camuffo et al., 2022).

1.2 Problem and instability in current methods

Three main approaches have emerged for tunnel point-cloud segmentation. Feature-engineering encodes domain knowledge into deterministic rules defined through geometric thresholds, line and edge features, curvature descriptors, or clustering criteria (Duda and Hart, 1972; Fischler and Bolles, 1981; Ester et al., 1996; Pauly et al., 2002). Because these rules are explicit and interpretable, engineers can readily audit and refine them, and such pipelines therefore remain widely used. However, they are inherently brittle: rules tuned to a specific tunnel geometry or scanning configuration often fail under changed conditions, requiring extensive manual reconfiguration (Weidner and Walton, 2024). Supervised deep learning reduces reliance on hand-crafted rules by learning hierarchical representations from large annotated datasets (Qi et al., 2017; Hu et al., 2020; Schult et al., 2023; Kolodiaznyh et al., 2024; Cha et al., 2024; Su et al., 2024). However, such models require large labelled datasets that are difficult to obtain in this domain, significant computational resources, and periodic retraining; their internal decision logic is also opaque, reinforcing engineers’ concerns about trustworthiness. Foundation models pre-trained on large and diverse datasets offer zero-shot generalisation guided by user prompts, reducing annotation demands (Kirillov et al., 2023; Bommasani et al., 2021). In civil engineering, such models have been applied to

tunnel component segmentation, structural health monitoring, and Scan-to-BIM workflows (R et al., 2024; Wang et al., 2024; Pan et al., 2023). Among them, SAM4Tun (Ye et al., 2025), built on Meta’s Segment Anything Model (SAM) (Kirillov et al., 2023), combines geometric preprocessing, image-based line detection, and prompt-based segmentation into a unified pipeline. SAM4Tun achieves strong performance under expert parameter tuning, yet remains highly sensitive to preprocessing and prompting choices. Deviations in tunnel geometry or scanning conditions from those assumed during tuning lead to sharp performance degradation.

Current methods for tunnel point-cloud segmentation therefore remain unstable when tunnel geometry and scanning conditions depart from the conditions under which they were tuned or trained. Feature-engineered pipelines are practical and interpretable but brittle and require repeated manual reconfiguration. Supervised deep learning reduces reliance on hand-crafted rules but depends on large labelled datasets, substantial computational resources, and retraining. Prompt-based foundation-model pipelines reduce annotation demand, yet their performance remains highly sensitive to preprocessing and prompting choices. Consistent segmentation performance across varying tunnel conditions remains difficult to achieve.

1.3 Why this matters

In practice, this instability causes repeated expert retuning, additional quality control, slower deployment across tunnel networks, and lower confidence in automation. When a single expert-tuned configuration is applied across tunnels with different diameters, ring lengths, segment counts, and scanning densities, segmentation quality can degrade severely — as demonstrated in our experiments where a fixed baseline achieves a mean mIoU of only 0.042 on complex tunnels compared with 0.367 on the regular tunnels for which it was tuned. Unstable segmentation can also lead to missed or mischaracterised defects in downstream inspection, assessment, and maintenance workflows.

1.4 Objective

We therefore develop R4Tun, an LLM-driven adaptation framework that augments an expert-designed tunnel segmentation pipeline so that stage parameters can be adjusted to changing tunnel conditions without expert tuning. The framework provides each LLM agent with structured context comprising memory (reference tunnel characteristics), state (intermediate pipeline outputs), and knowledge (domain-specific tuning guidelines), from which the agent produces bounded parameter updates via chain-of-thought reasoning. The paper evaluates whether this structured context can improve segmentation adaptability across regular and complex tunnels, and whether the adaptation behaviour is consistent across independent LLMs.

1.5 Contributions

This paper makes the following contributions:

1. A framework design in which LLM agents adapt the parameters of a fixed, expert-designed segmentation pipeline using structured context from memory, state, and domain knowledge, without modifying the pipeline’s algorithms.

2. A cumulative ablation methodology that isolates the incremental contribution of each context component (memory, state, knowledge) to segmentation performance across 30 tunnels.
3. Cross-LLM validation showing that three independent LLMs (Claude Opus 4.6, GPT-5.4, Gemini 3 Flash) produce statistically consistent adaptation patterns, converging on the same critical parameters and tunnel-family clustering.
4. A sensitivity analysis of 1,350 adapted parameter files identifying 11 tunnel-responsive and 7 baseline-correction parameters, with evidence that adaptation patterns are driven by objective tunnel characteristics rather than LLM-specific biases.

1.6 Paper organisation

Section 2 reviews related work on tunnel point-cloud segmentation, foundation models for infrastructure, and LLMs as reasoning agents in engineering tasks. Section 3 presents the materials and methods, including the R4Tun architecture, dataset, experimental design, evaluation metrics, and sensitivity analysis methodology. Section 4 reports results on main performance, ablation, sensitivity, error analysis, and practical considerations. Section 5 discusses findings, implications, and limitations. Section 6 concludes.

2. Related work

2.1 Tunnel point-cloud segmentation

Automated segmentation of tunnel linings from point-cloud data has followed two broad traditions. Feature-engineered approaches encode domain knowledge into deterministic geometric rules — thresholds on curvature, radius, line features, or clustering criteria (Duda and Hart, 1972; Fischler and Bolles, 1981; Ester et al., 1996; Pauly et al., 2002). Because these rules are explicit and auditable, they remain widely used in practice, particularly in safety-critical inspection workflows where interpretability is a requirement. However, they are inherently brittle: thresholds calibrated to one tunnel’s geometry, diameter, or scanning density often require extensive manual reconfiguration when conditions change (Weidner and Walton, 2024). Supervised deep-learning methods reduce reliance on hand-crafted rules by learning hierarchical representations from annotated point clouds (Qi et al., 2017; Hu et al., 2020; Schult et al., 2023; Kolodiazhnyi et al., 2024). These models have demonstrated strong segmentation performance under controlled conditions. However, they require large labelled datasets that are costly to obtain for tunnel linings, significant computational resources for training, and periodic retraining when deployed across new tunnel typologies. Furthermore, their internal decision logic remains opaque, which limits engineers’ ability to audit, override, or explain the system’s behaviour (Cha et al., 2024; Su et al., 2024).

2.2 Foundation models and prompt-based segmentation

Foundation models pre-trained on large and diverse datasets have recently been explored to mitigate annotation and retraining demands. The Segment Anything Model (SAM) (Kirillov et al., 2023) achieves zero-shot segmentation guided by point, box, or mask prompts, and has been applied to structural health monitoring, crack detection, and Scan-to-BIM workflows in civil engineering (R et al., 2024; Wang et al., 2024; Pan et al., 2023). For tunnel linings, SAM4Tun (Ye et al., 2025) combines geometric preprocessing

— unfolding the 3D tunnel into a 2D depth map, denoising, and enhancement — with Hough-transform-based joint detection and template-based SAM prompting. Under expert parameter tuning, SAM4Tun achieves competitive segmentation quality. However, performance depends critically on the values of approximately 60 pipeline parameters (thresholds, window sizes, sampling densities, merging criteria) that control preprocessing, prompt generation, and post-processing. When tunnel geometry or point-cloud quality deviates from the conditions assumed during tuning, these parameters become misspecified and segmentation quality degrades sharply. This sensitivity to parameter configuration limits the method’s adaptability and reproducibility beyond the narrow conditions for which it was calibrated.

2.3 LLMs as reasoning agents in engineering tasks

Advances in natural language processing have enabled large language models (LLMs) to be embedded in engineering workflows as general-purpose assistants (Bradley et al., 2023; Ghafarollahi and Buehler, 2024; Qian et al., 2024; Hong et al., 2024; Chen et al., 2024; Garcia et al., 2024). Reasoning-enabled LLMs incorporate post-training on curated, step-by-step reasoning traces, enabling them to articulate intermediate steps explicitly and improving logical consistency and verifiability (Wei et al., 2023; Kojima et al., 2023; Xiang et al., 2025). Multi-agent architectures further structure this capability: individual LLM agents are assigned specialised roles and coordinated through shared context to solve complex tasks (Hong et al., 2024; Chen et al., 2024; Liu et al., 2025; Gao et al., 2025). Context engineering — the systematic design of information provided to reasoning models — has been shown to strongly influence model performance (Xu et al., 2024; Mei et al., 2025; Anthropic, 2025; OpenAI, 2025). However, for construction and infrastructure applications, a key challenge remains: integrating such reasoning models in a way that enhances the generalisation of expert-designed pipelines while preserving the interpretability and overridability that engineers require.

2.4 Synthesis

Existing studies have shown that both supervised deep learning and foundation-model pipelines can achieve strong segmentation performance under controlled conditions. However, evidence remains difficult to compare because datasets, sensing setups, and validation conditions differ across studies. More importantly, most evaluations stop at model accuracy on a fixed test set and do not show whether performance remains reliable when tunnel geometry, diameter, ring structure, or scanning conditions change from the tuning reference. Feature-engineered pipelines are interpretable but lack adaptability; deep-learning models adapt through training data but lack interpretability; foundation-model pipelines reduce annotation demand but remain sensitive to parameter configuration. No existing approach combines adaptability with interpretability in a way that allows an expert-designed pipeline to adjust its parameters systematically to new tunnel conditions while keeping every decision traceable and overridable. This gap motivates the present study.

3. Materials and methods

3.1 Problem definition

The task is semantic segmentation of segmental tunnel linings from terrestrial laser scanning (TLS) point clouds. Given a raw point cloud of a tunnel section, the goal is to assign each point a structural label: background, key block (K), base blocks (B1, B2), and adjacent blocks (A1, A2, A3), with an additional A4 class for 7-segment complex tunnels. The intended downstream use is automated inspection — deformation assessment and defect localisation — where consistent, reliable segmentation across varying tunnel conditions is more important than peak accuracy on a single, well-tuned configuration.

3.2 Dataset

The experiments use the Seg2Tunnel benchmark, a publicly available TLS point-cloud dataset comprising five real tunnels scanned with a Leica C10 scanner. From these five tunnels, 30 contiguous subsets were extracted, each containing approximately 10 rings of tunnel lining (Table 1).

The 30 subsets span systematic variation along four structural dimensions:

Property	Regular tunnels (T1, T2)	Continuous tunnels (T3)	Complex tunnels (T4, T5)
Inner diameter	5.5 m	5.5 m	7.5 m
Ring length	1.2 m	1.2 m	1.8 m
Segments per ring	6	6	7
Joint type	Staggered	Continuous	Complex interleaved
Scanning	Single-station (Wuxi)	Multi-station registration	Single-station, offset centre (Fuzhou)
Evaluation schema	6-class	6-class	7-class
Count	10 subsets (1-1 to 2-5)	3 subsets (3-1-1 to 3-1-3)	17 subsets (4-1 to 5-7)

For reporting, regular and continuous tunnels are grouped as “regular” ($n = 13$), reflecting their shared 5.5 m diameter and 6-class schema. Complex tunnels form a separate group ($n = 17$). This grouping is used throughout.

The baseline sam4tun configuration was tuned by domain experts on a single reference tunnel (T2-2, “sample”) with diameter 5.60 m, density 2,466 pts/m³, and 10 rings. This reference achieves mIoU 0.531 under expert tuning.

3.3 Proposed method

3.3.1 Overview

R4Tun augments an expert-designed segmentation pipeline (SAM4Tun) with an LLM-driven adaptation layer. The pipeline’s five sequential stages — Unfolding, Denoising, Enhancing, Detecting, and SAM segmentation — remain fixed; only the parameter JSON files fed to each stage change. Each stage is managed by an LLM agent that receives structured context and produces bounded parameter updates via chain-of-thought (CoT) reasoning.

3.3.2 Pipeline stages

The five-stage pipeline transforms a raw 3D TLS point cloud into per-point semantic labels:

Stage 1 — Unfolding. Estimates the tunnel centreline using RANSAC-based ellipse fitting on cross-sectional slices, then maps each point to cylindrical coordinates (r , θ , h) to produce a 2D panoramic depth map. Key parameters: slice half-thickness (δ), slice spacing factor, vertical filter window, RANSAC settings, polynomial degree for centreline smoothing, and tunnel diameter.

Stage 2 — Denoising. Removes non-structural artefacts (rails, cables, scattered points) using grid-based radial-density filtering in cylindrical coordinates. A pair of radial masks constrains filtering to the expected tunnel radius. Key parameters: radial mask bounds (mask_r_low , mask_r_high), grid resolution (y_step , z_step), gradient threshold, smoothing window size, and default radial cutoff.

Stage 3 — Enhancing. Improves geometric continuity through curvature-guided point insertion and three-stage progressive upsampling, then projects the refined surface into a panoramic depth map with pixel-level interpolation. Key parameters: upsampling target distances (three stages), curvature threshold, depth thresholds, interpolation radius, and window size.

Stage 4 — Detecting. Applies Hough-transform-based line detection to the depth map to extract ring boundaries, then constructs template-based prompt coordinates for SAM. Key parameters: binary threshold, Hough thresholds (oblique, horizontal, vertical), line length and gap settings, merge distance, and ring spacing constant.

Stage 5 — SAM segmentation. Uses detected ring boundaries to construct template-based point and mask prompts, applies SAM (ViT-H) to produce 2D segment masks, and reprojects labels into 3D. For complex 7-segment tunnels, a geometric fallback replaces SAM. Key parameters: segment_per_ring , segment_order , segment dimensions, processing settings (resolution, padding, crop margin), and prompt point templates.

3.3.3 LLM adaptation layer

Each stage agent receives structured context comprising three components and produces a parameter JSON file:

Memory stores the reference tunnel’s characteristics (geometry, point density, coordinate ranges, nearest-neighbour distances) alongside the expert-tuned reference parameters from SAM4Tun. During reasoning, the agent compares the current tunnel’s characteristics against this reference to quantify deviation and anchor its parameter adjustments.

State captures the evolving geometric and statistical properties of the point cloud across processing stages. After each stage executes, a characteriser plugin extracts structured summaries (e.g., cylindrical coordinate percentiles after unfolding, retention rate and surface completeness after denoising, coverage uniformity after enhancing, prompt distribution after detecting). These cumulative state summaries are injected into subsequent agents’ prompts, enabling each agent to reason about how prior stages have transformed the data.

Knowledge supplies domain-specific guidance in the form of parameter definitions, adaptation rules, proven defaults, and tunnel-family classification criteria. Each stage has its own knowledge document

covering: tunnel-type taxonomy (T1–T5 variations in diameter, ring length, segment count, joint type, and scanning configuration); parameter semantics and empirically validated ranges; classification criteria for tunnel conditions (SIMILAR, DENSE, SPARSE, LARGE-DIAMETER, CHALLENGING); and diagnostic rules linking observed characteristics to parameter adjustments.

The agent’s reasoning follows a structured CoT: (1) anchoring — quantifying deviation from the reference; (2) classification — categorising the tunnel condition; (3) diagnostic inspection — identifying which parameters are implicated; (4) parameter adaptation — proposing bounded updates with evidence; and (5) validation — checking logical consistency of the proposed changes.

3.3.4 Implementation

For each tunnel and condition, the orchestrator runs a sequential loop: for each of the five stages, it (a) builds the LLM prompt from the current context level, (b) calls the LLM API, (c) extracts the JSON parameter response, (d) executes the pipeline stage with those parameters, and (e) runs the characteriser plugin to update the state for the next stage. Each stage’s parameters are inferred independently from the reference baseline — the LLM does not see its own prior-stage parameter outputs, preventing self-reinforcement of errors. The five stage scripts and evaluation code are identical across all conditions; only the parameter JSONs change.

3.4 Baseline

The baseline is the fixed SAM4Tun configuration (“sam4tun”), in which a single set of expert-tuned parameters is applied to all 30 tunnels without any per-tunnel adaptation. This represents the standard deployment scenario where an engineer tunes parameters on one reference tunnel and applies them uniformly.

3.5 Experimental design

3.5.1 Ablation ladder

The experiment follows a cumulative ablation design with four conditions, each adding one context component:

Level	Code	Condition	What the LLM sees
0	sam4tun	Baseline (fixed params)	Nothing — fixed default parameters for all tunnels
1	m	Memory	Reference tunnel characteristics and reference parameters
2	m_s	Memory + State	+ intermediate pipeline stage outputs (cumulative characteristics)
3	m_s_k	Memory + State + Knowledge	+ domain knowledge (parameter semantics, adaptation rules, tunnel-family taxonomy)

This cumulative design means each level includes all components from lower levels, isolating the incremental contribution of each new component.

3.5.2 Cross-LLM validation

Each ablation condition was run with three independent LLMs to assess whether adaptation behaviour depends on the specific model:

- **Claude Opus 4.6** (Anthropic) — reasoning-enabled model with strong analytical capabilities
- **GPT-5.4** (OpenAI) — coding and numerical parameter generation
- **Gemini 3 Flash** (Google) — fast inference with multi-modal capabilities

All three LLMs received identical prompts and context for each condition. The pipeline scripts and evaluation code were shared across all runs.

3.5.3 Statistical testing

For each condition and LLM, paired differences $\Delta_i = \text{mIoU}_{\text{condition}_i} - \text{mIoU}_{\text{baseline}_i}$ were computed for each tunnel i . Mean, standard deviation, and two-sided paired t-test p-values were computed per tunnel family (regular $n = 13$, complex $n = 17$) and overall ($n = 30$). Significance was assessed at $\alpha = 0.05$.

As a non-parametric sensitivity check (paired differences need not be normal at $n = 13$ or $n = 17$), the same paired comparisons were repeated with two-sided Wilcoxon signed-rank tests on the per-tunnel mIoU pairs. Full tables (t-test vs Wilcoxon for overall, regular $\cup$ continuous, alternated, continuous, and complex subsets) are in [methods/papers/output/wilcoxon_vs_ttest.md](#). For **memory+state** and **memory+state+knowledge** vs baseline, Wilcoxon p-values agree with the t-test conclusions on overall and complex subsets (both $p < 0.0001$ across all three LLMs). On the small continuous subset ($n = 3$), Wilcoxon p-values are uninformative (e.g. $p = 0.25$) where the t-test remains suggestive; conclusions for continuous tunnels should therefore be interpreted cautiously. Tunnel-level bootstrap confidence intervals on mean paired differences (10 000 resamples of the 30 tunnels) and sign tests for the knowledge increment are reported in [methods/papers/output/confidence_analysis.md](#).

3.6 Evaluation metrics

Segmentation quality is measured by mean Intersection-over-Union (mIoU):

$$\text{IoU}_c = \text{TP}_c / (\text{TP}_c + \text{FP}_c + \text{FN}_c), \text{mIoU} = (1/C) \sum \text{IoU}_c$$

where C is the number of classes (6 for regular/continuous, 7 for complex tunnels) and TP_c , FP_c , FN_c are true positive, false positive, and false negative counts for class c . The evaluation schema is determined automatically from the ground-truth labels: max GT label > 6 triggers 7-class evaluation. Overall accuracy (OA) and macro F1 are also computed but mIoU is the primary metric as it directly penalises both missing regions and spurious predictions and is less sensitive to class imbalance.

3.7 Sensitivity analysis

To assess whether the claim of improved adaptability is supported by evidence beyond aggregate mIoU, three complementary sensitivity analyses were conducted:

3.7.1 Critical parameter identification

All 1,350 adapted parameter files (30 tunnels $\times$ 3 LLMs $\times$ 3 ablation conditions $\times$ 5 stages) were compared against the fixed SAM4Tun baseline. For each (stage, parameter) pair, the analysis recorded: how many of the 30 tunnels triggered a change, how many LLMs independently adapted that parameter, the coefficient of variation (CV) of adapted values across tunnels, and whether the adaptation was tunnel-responsive ($CV \geq 0.06$, value varies per tunnel) or a baseline correction ($CV \approx 0$, same corrected value for all tunnels). Parameters adapted in $\geq 28/30$ tunnels across all 3 LLMs were classified as “always-trigger” critical parameters.

3.7.2 Cross-LLM consistency

For each always-trigger parameter, per-LLM tunnel counts and value ranges were extracted to confirm that all three LLMs adapt the same parameter keys and produce the same tunnel-family clustering (e.g., families 1-x and 2-x cluster for mask_r_low; families 4-x and 5-x form a separate cluster). The adapted values differ slightly between models, but the patterns — which parameters change, which tunnels cluster together, which direction the change goes — were compared for consistency.

3.7.3 Characteristic-to-parameter correlation

Spearman rank correlations ($N = 30$ tunnels) were computed between every numeric characteristic field visible to each stage and every adapted parameter value. Fields with $|\rho| \geq 0.5$ and $p < 0.05$ were flagged as significant drivers. This was cross-validated with text-mining of LLM reasoning traces to confirm that statistically significant characteristics were also explicitly referenced in the agents’ CoT outputs.

3.8 Limitations and quality of evidence

Several methodological limitations constrain the confidence that can be placed in the findings:

Single pipeline architecture. R4Tun was evaluated on SAM4Tun only. Whether the same adaptation framework generalises to other segmentation pipelines is untested.

Ground-truth dependence. mIoU evaluation requires manual annotations (segment and ring labels in the Seg2Tunnel dataset). The framework itself is GT-free at runtime, but evaluation quality depends on annotation accuracy.

Single reference configuration. All adaptation is anchored to one expert-tuned reference tunnel (T2-2). Tunnels that deviate strongly from this reference in multiple dimensions simultaneously (e.g., OR-2 with both large diameter and low density) receive weaker anchoring.

SAM non-determinism. The SAM segmentation stage involves GPU computation that can produce slightly different outputs across runs. A rerun of the same configuration on tunnel 1-1 showed mIoU varying from 0.585 to 0.559 ($\Delta = 0.026$), indicating that reported mIoU values carry run-to-run noise of approximately ± 0.03 .

LLM stochasticity. LLM outputs are inherently stochastic. Each condition was run once per LLM; repeated trials would provide confidence intervals on adaptation quality but were not conducted due to API cost constraints.

Three LLMs only. Cross-model validation covers three commercial LLMs. Open-source or smaller models were not tested; adaptation quality may differ.

No alternative optimisation comparison. The study compares LLM adaptation against a fixed baseline but not against other automated parameter tuning methods (e.g., grid search, Bayesian optimisation) applied to the same search space.

Quality of the underlying evidence. The evidence base varies in strength across the study’s findings:

- *Aggregate mIoU improvement (high quality):* $n = 30$ paired design, consistent across three independent LLMs, confirmed by both parametric (t-test) and non-parametric (Wilcoxon) tests, $p < 0.0001$ for memory+state and memory+state+knowledge conditions.
- *Component decomposition (moderate–high):* Cumulative ablation across 3 LLMs consistently ranks state > knowledge > memory, but the cumulative design cannot isolate pairwise interaction effects.
- *Cross-LLM convergence (high):* 1,350 parameter files from three independent LLMs show convergent parameter keys, values, and tunnel-family clustering. However, based on a single run per LLM per condition.
- *Characteristic-to-parameter correlation (moderate):* Spearman correlations ($N = 30$) with cross-validation against CoT text-mining. Limited by single-run parameter values and 30-tunnel sample size.
- *Memory-only effect (low):* Small, inconsistent across LLMs (-0.006 to $+0.049$), statistically non-significant for two of three LLMs at $\alpha = 0.05$.
- *Knowledge incremental effect (low–moderate):* Mean increment on top of memory+state is $+0.011$ to $+0.022$; **tunnel-bootstrap 95% CIs on that mean straddle zero for all three LLMs** (see [methods/papers/output/confidence_analysis.md](#)). One-sided binomial tests on per-tunnel sign of the increment are significant for Opus and GPT at $n = 30$ (21/30 positive, $p = 0.021$) and for complex subsets on GPT (15/17, $p = 0.001$) and Gemini (14/17, $p = 0.006$).
- *Tunnel-bootstrap uncertainty (moderate):* Non-parametric CIs from resampling the 30 tunnels address **benchmark composition** uncertainty; they do not quantify SAM rerun or LLM API stochasticity.
- *Per-class IoU and spread analysis (moderate):* Aggregated from existing evaluation files; spread statistics recomputed in `confidence_analysis.py`.

A structured per-claim confidence assessment is provided in Section 5.5.

4. Results

4.1 Main quantitative results

Table 2 summarises the overall and per-family segmentation performance across all four conditions and three LLMs.

Table 2: Mean mIoU by condition and LLM ($n = 30$ tunnels)

Condition	Opus 4.6	GPT-5.4	Gemini 3 Flash
sam4tun (baseline)	0.150	0.150	0.150
memory	0.144	0.182	0.199
memory+state	0.312	0.299	0.302
memory+state+knowledge	0.328	0.321	0.313

Table 3: Paired differences vs baseline — Overall (n = 30)

Mean paired Δ mIoU, two-sided paired t-test p-value, and tunnel-bootstrap 95% CI (10 000 resamples; see [methods/papers/output/confidence_analysis.md](#)).

Condition	Opus 4.6	GPT-5.4	Gemini 3 Flash
memory	−0.006 (p = 0.558) CI [−0.027, 0.014]	+0.032 (p = 0.028) CI [0.005, 0.058]	+0.049 (p = 0.056) CI [0.006, 0.101]
memory+state	+0.162 (p < 0.0001) CI [0.126, 0.198]	+0.149 (p < 0.0001) CI [0.110, 0.190]	+0.152 (p < 0.0001) CI [0.117, 0.189]
memory+state+knowledge	+0.178 (p < 0.0001) CI [0.139, 0.216]	+0.171 (p < 0.0001) CI [0.140, 0.203]	+0.163 (p < 0.0001) CI [0.122, 0.203]

For memory+state and memory+state+knowledge, all bootstrap CIs exclude zero across all three LLMs, confirming the headline finding under tunnel-composition uncertainty. For memory alone, the CI for Opus 4.6 straddles zero while GPT-5.4 and Gemini 3 Flash CIs are marginally positive.

Table 4: Mean mIoU by tunnel family and condition (best LLM per cell)

Family	n	sam4tun	memory	memory+state	m+s+k
Regular (alternated)	10	0.367	0.312–0.333	0.590–0.607	0.586–0.621
Continuous	3	0.038	0.087–0.384	0.239–0.333	0.192–0.251
Complex	17	0.042	0.055–0.101	0.126–0.155	0.169–0.177
Regular (all, n = 13)	13	0.291	0.260–0.344	0.516–0.531	0.495–0.535
Overall	30	0.150	0.144–0.199	0.299–0.312	0.313–0.328

The fixed baseline achieves reasonable performance on regular alternated tunnels (mean mIoU 0.367) for which it was tuned but degrades severely on continuous tunnels (0.038) and complex tunnels (0.042). The full R4Tun design (m+s+k) improves all families substantially, with the largest relative gains on complex tunnels where the baseline is weakest.

Performance distribution across tunnels

To characterise how adaptation affects the full performance distribution rather than only the mean, Table 4a reports the standard deviation and min–max range of per-tunnel mIoU under each condition, averaged across the three LLMs.

Table 4a: Performance spread across 30 tunnels (mean across 3 LLMs)

Values computed from per-tunnel mIoU in the comparison journals; see [methods/papers/output/confidence_analysis.md](#).

Metric	sam4tun	memory	memory+state	m+s+k
Mean mIoU	0.150	0.175	0.304	0.320
Std of per-tunnel mIoU	0.166	0.136	0.228	0.218
Min tunnel mIoU	0.032	0.042	0.082	0.072
Max tunnel mIoU	0.532	0.471	0.682	0.679
Δ min mIoU vs baseline (floor lift)	—	+0.010	+0.050	+0.040

Within-family std of per-tunnel mIoU (mean across 3 LLMs)

Family	n	sam4tun	memory	memory+state	m+s+k
Regular $\cup$ continuous	13	0.168	0.121	0.170	0.190
Complex	17	0.003	0.032	0.047	0.074

The baseline exhibits bimodal behaviour: regular tunnels cluster at higher mIoU while complex tunnels cluster near 0.04, producing high overall standard deviation relative to the mean ($CV \approx 1.1$). The mean min tunnel mIoU rises from 0.032 (baseline) to 0.082 under memory+state and 0.072 under m+s+k (row Δ min mIoU reports mean across LLMs of $\min(\text{condition}) - \min(\text{baseline})$). Overall per-tunnel std increases under adapted conditions because successful adaptation pulls regular tunnels to much higher mIoU while complex tunnels remain lower, widening the between-family gap. Within the regular $\cup$ continuous family, per-tunnel std is similar before and after full adaptation (0.168 $\rightarrow$ 0.190). Within complex tunnels, baseline std is artificially tiny (most tunnels fail similarly near 0.04); adaptation increases within-family std (0.003 $\rightarrow$ 0.074) as some tunnels gain much more mIoU than others — a sign of differentiated recovery, not uniform collapse.

Per-class IoU breakdown

To examine whether mIoU gains reflect uniform improvement across segment types or concentration in a few classes, mean per-class IoU was aggregated from existing `evaluation/performance.md` files across the three LLM ablation trees. Table 4b shows the Opus 4.6 results (representative; GPT-5.4 and Gemini 3 Flash show the same pattern).

Table 4b: Mean per-class IoU by family and condition (Opus 4.6)

Regular $\cup$ continuous ($n = 13$, 6-class schema):

Class	sam4tun	memory	memory+state	m+s+k
Background	0.640	0.559	0.741	0.751
K-block	0.192	0.129	0.373	0.386
B1-block	0.261	0.206	0.527	0.540
A1-block	0.253	0.276	0.537	0.542
A2-block	0.150	0.159	0.380	0.420
A3-block	0.286	0.259	0.519	0.550
B2-block	0.256	0.232	0.534	0.558

Complex ($n = 17$, 7-class schema):

Class	sam4tun	memory	memory+state	m+s+k
Background	0.337	0.358	0.513	0.520
K-block	0.000	0.034	0.183	0.159
B1-block	0.000	0.001	0.084	0.135
A1-block	0.000	0.005	0.128	0.155
A2-block	0.000	0.006	0.143	0.116
A3-block	0.000	0.017	0.092	0.119
A4-block	0.000	0.019	0.100	0.104
B2-block	0.000	0.000	0.000	0.043

Three patterns emerge. First, for regular tunnels, all structural classes improve roughly uniformly from baseline to m+s+k (doubling or more), with K-block and A2-block showing the largest relative gains. Second, for complex tunnels, the baseline assigns nearly all points to background (all segment-class IoUs are 0.000); adaptation recovers segment structure progressively, with knowledge adding the most for B1, A1, and A3. Third, **B2-block for complex tunnels remains at 0.000 under memory and memory+state and rises to only 0.028–0.095 under m+s+k** — this is the hardest class to recover and specifically requires the tunnel-family-specific guidance in the knowledge component (7-segment layout, B2 crop geometry, 7.5/5.5 scaling). Full tables for all three LLMs are in [methods/papers/output/per_class_iou_summary.md](#).

4.2 Ablation and component analysis

The cumulative ablation isolates the incremental contribution of each context component. Table 5 presents the mean delta vs baseline for each step across the three LLMs.

Table 5: Incremental mIoU contribution of each context component (mean delta vs previous level)

Transition	Opus 4.6	GPT-5.4	Gemini 3 Flash
Baseline → Memory	−0.006	+0.032	+0.049
Memory → Memory+State	+0.168	+0.117	+0.103
Memory+State → M+S+K	+0.016	+0.022	+0.011

Memory alone provides an initial anchor but produces inconsistent results. For Opus 4.6, memory alone slightly degrades overall performance (-0.006 , $p = 0.558$). For GPT-5.4, the gain is small but significant ($+0.032$, $p = 0.028$). For Gemini, the gain is borderline ($+0.049$, $p = 0.056$). On regular alternated tunnels specifically, memory alone tends to degrade performance across all three LLMs (-0.055 to -0.035 for alternated tunnels), while providing some benefit for continuous tunnels where the baseline is near-zero. Memory alone is a passive component: it preserves access to reference configurations but without state updates or domain knowledge, the agent cannot reason about how the current tunnel deviates from that reference or how parameters should respond.

State is the dominant driver of improvement. Adding intermediate pipeline characteristics to the context produces large, consistent gains across all three LLMs and all tunnel families (all $p < 0.0001$ overall). The incremental delta from adding state ranges from $+0.103$ (Gemini) to $+0.168$ (Opus), with paired Cohen’s d of 1.32 – 1.61 across the three LLMs — a “very large” effect by conventional thresholds ($d > 0.8$). This is because state provides the agent with explicit quantitative evidence of how each pipeline stage has transformed the data — radial percentiles after unfolding, retention rates after denoising, coverage uniformity after enhancing, and prompt distribution after detecting. With this information, the agent can set parameters such as radial mask bounds directly from measured percentiles rather than guessing from raw characteristics alone.

Knowledge adds a further, smaller but consistent improvement. Adding domain knowledge on top of memory and state produces an additional delta of $+0.011$ to $+0.022$ overall (paired Cohen’s $d = 0.11$ – 0.31 , “small” by convention). **Tunnel-bootstrap 95% confidence intervals on that mean increment straddle zero for every LLM** (see [methods/papers/output/confidence_analysis.md](#)), so the *mean* knowledge effect is not significantly bounded away from zero under resampling of tunnels alone; nevertheless, **21 of 30 tunnels** show a strictly positive increment for Opus 4.6 and GPT-5.4 (one-sided binomial vs fair coin, $p = 0.021$), and **19 of 30** for Gemini 3 Flash ($p = 0.10$). On complex tunnels only, **15/17** (GPT) and **14/17** (Gemini) are positive ($p = 0.001$ and $p = 0.006$). The gain is most pronounced on complex tunnels, where knowledge provides tunnel-family-specific guidance (e.g., `ring_spacing_constant` = 1.8 for T4/T5 vs 1.2 for T1–T3; 7 segments per ring instead of 6; scaling prompt templates by $7.5/5.5$). For regular tunnels where the baseline already performs reasonably after state adaptation, knowledge contributes smaller marginal improvements or, in some cases, slightly reduces performance by introducing conservative constraints.

4.3 Sensitivity results

4.3.1 Critical parameter identification

Analysis of 1,350 adapted parameter files identified 18 “always-trigger” parameters that all three LLMs independently adapt for nearly all tunnels ($\geq 28/30$). These fall into two distinct categories:

Table 6: Tier 1 — Tunnel-responsive parameters (11 parameters, $CV \geq 0.06$)

Stage	Parameter	Tunnels	CV	Baseline	Adapted range	Physical driver
Denoising	mask_r_low	30/30	0.082	2.7	[2.09, 3.75]	Tunnel inner radius
Denoising	mask_r_high	30/30	0.147	2.8	[2.78, 4.38]	Tunnel outer radius
Denoising	default_cutoff_z	29/30	0.142	2.7	[2.65, 6.27]	Radial extent
Denoising	z_step	30/30	0.181	0.001	[0.003, 0.005]	Scanner resolution
Detecting	hough_threshold_oblique	30/30	0.188	50	[20, 83]	Point density
Detecting	hough_threshold_horizontal	30/30	0.204	50	[20, 83]	Point density
Detecting	hough_threshold_vertical	28/30	0.219	500	[320, 980]	Ring spacing
Enhancing	inter_radius	30/30	0.130	0.06	[0.03, 0.08]	Mean point spacing
Enhancing	upsampling_stage1	30/30	0.064	0.08	[0.055, 0.11]	Density regime
Unfolding	diameter	27/30	0.072	5.5	[5.31, 7.6]	Physical diameter
SAM	processing.padding	29/30	0.265	150	[160, 419]	Segment width

Tunnel-responsive parameters exhibit clear clustering by tunnel family. For example, mask_r_low maps directly to physical radius: families 1-x and 2-x receive 2.25–2.38, while families 4-x and 5-x receive 2.62–2.91. The Hough thresholds inversely track point density: sparse tunnels (family 5) receive lower thresholds (30–42) to detect faint lines, while dense tunnels (families 1-2) receive higher thresholds (53–61) to filter noise.

Table 7: Tier 2 — Baseline corrections (7 parameters, CV $\approx$ 0)

Stage	Parameter	Baseline	Corrected to	Shift
Denoising	smoothing_window_size	3	5	+67%
Denoising	smoothing_offset	−0.003	−0.002	+33%
Denoising	grad_threshold	0.2	0.15	−25%
Denoising	y_step	0.5	0.4	−20%
Enhancing	curvature_threshold	0.0005	0.005	+900%
Enhancing	depth_threshold_low	0.003	0.005	+67%
Enhancing	depth_threshold_high	0.008	0.015	+87%

All three LLMs independently converge on the same corrected values for these parameters, regardless of tunnel. This indicates that the SAM4Tun baseline contains approximately seven suboptimal defaults that any LLM will fix upon first exposure to the pipeline code and domain knowledge.

4.3.2 Per-stage adaptation coverage

Stage	Baseline params	Adapted	Coverage	Top parameter
Unfolding	16	10	63%	diameter (27/30 tunnels)
Denoising	8	8	100%	mask_r_low (30/30)
Enhancing	14	12	86%	inter_radius (30/30)
Detecting	14	14	100%	hough_threshold_oblique (30/30)
SAM	~50	45	90%	processing.padding (29/30)

Denoising and detecting are fully covered (every parameter is adapted); unfolding is the most conservative stage (63%), consistent with geometric unfolding being well-constrained by RANSAC.

4.3.3 Cross-LLM consistency

Despite different architectures, all three LLMs show near-identical adaptation patterns:

Table 8: Per-LLM adaptation summary

LLM	Unfolding	Denoising	Enhancing	Detecting	SAM	Total changes
Gemini 3 Flash	8 params, 22/30 tunnels	8, 30/30	11, 30/30	14, 30/30	44, 29/30	3,925
GPT-5.4	8, 26/30	8, 30/30	12, 30/30	14, 30/30	45, 30/30	3,615
Opus 4.6	9, 20/30	8, 30/30	10, 30/30	14, 30/30	44, 29/30	3,965

The parameter keys each LLM adapts are highly consistent; the adapted values differ slightly but track the same per-tunnel trends and produce the same tunnel-family clustering. This convergence across independent models — with no shared training data or coordination — provides evidence that the adaptations are driven by objective tunnel characteristics rather than LLM-specific biases.

4.3.4 Characteristic-to-parameter correlation

Spearman correlation analysis identified 38 cumulative characteristic fields across the five stages that significantly drive parameter adaptation. The strongest signals include:

- Unfolded r_percentiles p10 and p99 show $|\rho| = 1.0$ with mask_r_low and mask_r_high respectively, confirming that LLMs set radial masks directly from measured percentiles.
- Estimated diameter correlates at $|\rho| \approx 0.91$ with parameters across all five stages.
- Mean nearest-neighbour distance correlates at $|\rho| \approx 0.87$ with spacing-related parameters.

Text-mining of reasoning traces confirmed that fields with high statistical correlation are also explicitly referenced in the agents’ CoT outputs, establishing both quantitative and qualitative evidence of characteristic-driven adaptation.

4.4 Error analysis and failure cases

Despite overall improvements, several systematic weaknesses remain:

Memory-alone degradation on regular tunnels. For alternated regular tunnels (1-x, 2-x), memory alone degrades mIoU by -0.035 to -0.055 depending on the LLM. Without state or knowledge, the agent has only raw characteristics and reference parameters. When the current tunnel is similar to the reference, the agent still attempts to adapt parameters but lacks sufficient context to make informed changes, often pushing parameters away from their already-adequate baseline values.

Continuous tunnel variability. The three continuous tunnels (3-1-1, 3-1-2, 3-1-3) show high variance across LLMs and conditions. For example, under memory alone, Gemini achieves 0.384 on continuous tunnels while Opus achieves only 0.087. This reflects the small sample size ($n = 3$) and the distinctive characteristics of continuous-joint tunnels (multi-station registration, uniform density, different detection geometry) that are poorly represented in the knowledge base.

Complex tunnel ceiling. While the relative improvement on complex tunnels is large (+302–321%), the absolute mIoU remains low (0.169–0.177). Complex tunnels combine larger diameter, different ring length, higher segment count, single-station offset scanning, and density variation between near and far sides. These compounding challenges limit how much parameter adaptation alone can recover.

Per-tunnel failures. Specific tunnels where adaptation fails or produces minimal gain: - Tunnel 1-4: Gemini m_s_k achieves only 0.209 (below baseline 0.348), a rare case of degradation under the full design. - Tunnel 4-4: Opus m_s_k achieves only 0.047 (near baseline 0.042), indicating near-complete failure of adaptation. - Tunnel 5-4: Best m_s_k across LLMs is 0.122, the lowest among complex tunnels, indicating persistent difficulty.

4.5 Practical performance

LLM inference: Each full ablation run ($30 \text{ tunnels} \times 5 \text{ stages}$) requires 150 LLM API calls. Wall-clock time for a complete 30-tunnel run is approximately 5 hours, including both LLM inference and pipeline execution. API timeout is set to 300 seconds per call.

Pipeline execution: The deterministic pipeline stages (unfolding through SAM) run on a single NVIDIA RTX 4090 GPU. No retraining is required; the LLM adaptation layer operates purely through prompt-based inference.

Deployment model: R4Tun requires no labelled training data, no model fine-tuning, and no GPU training. The only requirement beyond the base SAM4Tun pipeline is API access to an LLM. Parameter adaptation is a one-time operation per tunnel: once parameters are inferred, the pipeline can be run repeatedly with those parameters without further LLM calls. All reasoning traces and parameter updates are logged in human-readable format for engineer review.

5. Discussion

5.1 Interpretation of findings

The results support the claim that LLM-driven parameter adaptation improves segmentation adaptability across varying tunnel conditions. We define “improved adaptability” here as: the adapted pipeline achieves significantly higher mean mIoU than the fixed baseline across tunnel families that differ from the reference configuration, consistently across three independent LLMs. The evidence supports this

definition — mean mIoU improves for every tunnel family and every LLM — but does not demonstrate reduced performance variance across tunnels (inter-family spread is preserved because regular tunnels improve more in absolute terms than complex tunnels). The claim is therefore about lifting performance on unseen configurations, not about tightening the performance distribution. Three findings are central:

First, **state is the dominant driver of improvement**, contributing +0.103 to +0.168 mIoU depending on the LLM, compared with -0.006 to $+0.049$ for memory alone and $+0.011$ to $+0.022$ for knowledge on top of state. This aligns with the design rationale: state provides explicit quantitative evidence of how each pipeline stage has transformed the data, enabling the agent to set parameters from measured statistics (e.g., radial percentiles for mask bounds) rather than guessing from raw characteristics alone. Without state, the agent has only pre-pipeline statistics that may not reflect how the tunnel presents after unfolding, denoising, or enhancing.

Second, **memory alone is insufficient and can be harmful**. On regular alternated tunnels, memory alone degrades performance by -0.035 to -0.055 . This suggests that providing an LLM with reference characteristics and parameters without intermediate feedback encourages premature or misdirected adaptation: the agent attempts to adjust parameters for perceived differences but cannot verify whether those adjustments improve or degrade the intermediate outputs. Memory is a necessary scaffold — it provides the baseline from which adaptation departs — but it requires state feedback to become effective.

Third, **knowledge provides targeted rather than uniform improvement**. The knowledge component adds the smallest incremental gain overall but is most valuable for complex tunnels, where it supplies tunnel-family-specific configuration (7-segment layout, 1.8 m ring spacing, 7.5/5.5 scaling factors) that neither memory nor state can provide. For regular tunnels already well-served by state-based adaptation, knowledge contributes marginal or sometimes slightly negative effects, suggesting that additional domain constraints can occasionally over-constrain an already adequate adaptation.

5.2 Comparison with prior work

R4Tun differs from prior tunnel segmentation approaches in its mechanism of adaptation. Feature-engineered pipelines (Weidner and Walton, 2024) require manual reconfiguration when conditions change; R4Tun automates this process. Supervised deep-learning methods (Qi et al., 2017; Hu et al., 2020; Schult et al., 2023) adapt through training data but require labelled datasets and retraining; R4Tun requires neither. Foundation-model pipelines like SAM4Tun (Ye et al., 2025) achieve strong performance under expert tuning but degrade when parameters are misspecified; R4Tun specifically addresses this degradation.

Compared with general LLM agent frameworks (Hong et al., 2024; Qian et al., 2024), R4Tun is constrained by design: agents operate within a fixed pipeline, adjust only numeric parameters within bounded ranges, and produce logged reasoning traces. This constrains the agent’s action space to prevent hallucination-driven parameter choices while preserving the interpretability and overridability that infrastructure engineers require.

The cross-LLM convergence finding — that three independent LLMs adapt the same parameters to the same tunnel-family clusters — is notable. It suggests that the adaptation patterns are not artefacts of a specific model’s training data or reasoning biases but reflect objective relationships between tunnel characteristics and pipeline parameters. This provides stronger evidence for the adaptability of the framework than single-model evaluation alone.

5.3 Implications for AIC readers

For practitioners in automated infrastructure inspection, R4Tun offers a deployment model in which an expert tunes a pipeline on one reference tunnel, encodes domain knowledge into human-readable documents, and then deploys the LLM adaptation layer to handle new tunnels without further manual intervention. The key practical benefits are:

- **Reduced retuning effort:** Parameter adaptation is automated for each new tunnel, eliminating the need for an expert to manually adjust thresholds and settings.
- **Preserved interpretability:** Every parameter change is linked to an explicit reasoning trace that engineers can inspect, override, or use to refine the knowledge base.
- **Graceful degradation:** Even when adaptation is imperfect (e.g., on complex tunnels where absolute mIoU remains modest), the framework avoids the catastrophic collapse seen with fixed parameters (baseline mIoU 0.042 on complex tunnels).
- **No retraining:** The approach requires no labelled training data, no model fine-tuning, and no GPU training infrastructure.

5.4 Limitations

Several limitations should be explicitly acknowledged:

1. **Single pipeline:** R4Tun was evaluated on SAM4Tun only. Transferability to other segmentation pipelines is untested.
2. **Single reference:** All adaptation is anchored to one expert-tuned reference. Tunnels deviating strongly in multiple dimensions simultaneously receive weaker anchoring and lower absolute performance.
3. **No alternative optimisation comparison:** The study does not compare LLM adaptation against Bayesian optimisation, grid search, or other automated tuning methods applied to the same parameter space and evaluation metric.
4. **Run-to-run variance:** SAM non-determinism introduces approximately ± 0.03 mIoU noise. Tunnel-level bootstrap CIs ([methods/papers/output/confidence_analysis.md](#)) quantify uncertainty over the 30 subsets, not repeated pipeline or LLM draws.
5. **LLM API dependency:** The framework requires commercial LLM API access, introducing cost, latency, and availability constraints not present in a standalone pipeline.
6. **Continuous tunnel under-representation:** Only 3 of 30 subsets are continuous tunnels, limiting statistical power for this family.
7. **Complex tunnel ceiling:** Absolute mIoU on complex tunnels (0.169–0.177) remains low, suggesting that parameter adaptation alone cannot fully compensate for the compounding challenges of large diameter, offset scanning, and density variation.

5.5 Confidence assessment

Table 9 rates confidence in each major claim, following the AIC requirement that model-based claims include assessments of confidence in the whole analysis.

Table 9: Per-claim confidence assessment

Bootstrap CIs: tunnel-level resampling of the 30 paired differences (10 000 draws); see [methods/papers/output/confidence_analysis.md](#). They quantify uncertainty over **which tunnels** are in the benchmark, not SAM or LLM rerun noise.

Claim	Supporting evidence	Bootstrap 95% CI (mean paired Δ)	Confidence	Key caveat
LLM adaptation raises mean mIoU vs fixed baseline	n = 30 paired design, 3 LLMs, $p < 0.0001$ (t-test and Wilcoxon) for m+s and m+s+k	m+s+k – baseline: all three LLMs strictly > 0 (Opus [0.139, 0.216], GPT [0.140, 0.203], Gem [0.122, 0.203])	High	± 0.03 SAM run-to-run noise not in CI
State is the dominant context component	Largest incremental delta (+0.103 to +0.168) across all 3 LLMs; all $p < 0.0001$	m+s – baseline: all three > 0 (e.g. [0.110, 0.190] GPT)	High	Cumulative design; interaction effects not isolated
Memory alone is insufficient	Cross-LLM pattern: degrades regular alternated tunnels; discovery, not a flaw	memory – baseline: Opus [−0.027, 0.014], GPT [0.005, 0.058], Gem [0.006, 0.101]	Moderate	Run-to-run / API stochasticity not in CI
Knowledge adds targeted improvement	Mean increment +0.011 to +0.022 on top of m+s	m+s+k – m+s: all three CIs straddle zero (Opus [−0.003, 0.034], GPT [−0.011, 0.049], Gem [−0.028, 0.046]); 21/30 tunnels strictly improve for Opus & GPT (one-sided binomial $p = 0.021$), 19/30 for Gem ($p = 0.10$); complex-only 15/17 (GPT, $p = 0.001$), 14/17 (Gem, $p = 0.006$)	Low–Moderate	Mean increment overlaps SAM noise; tunnel-bootstrap does not reject zero for any LLM
Adaptations driven by tunnel characteristics, not LLM biases	3 independent LLMs converge on same 18 critical parameters and tunnel-family clusters; Spearman $\rho \geq 0.87$ for key characteristic–parameter pairs	—	High	Single run per LLM; no open-source or smaller models tested
Per-class improvement is broad, not concentrated in one class	All 6–7 structural classes improve for regular tunnels; all recover from 0.000 for complex	—	High	B2-block for complex tunnels remains very low (0.028–0.095)
Complex tunnel ceiling exists	Best absolute mIoU 0.169–0.177 despite +302–321% relative gain	—	High (that ceiling exists)	May reflect pipeline architectural limits rather than adaptation limits

Overall confidence in the analysis: The central claim — that structured context improves pipeline adaptability across varying tunnels — is supported at high confidence by aggregate mIoU (paired design, three LLMs, Wilcoxon agreement, and tunnel-bootstrap CIs for m+s and m+s+k vs baseline that exclude zero for every LLM). Memory’s cross-LLM inconsistency is a substantive finding (moderate confidence).

Knowledge contributes a small mean increment whose tunnel-bootstrap CI straddles zero for all three LLMs (low–moderate confidence), but **directional consistency** (majority of tunnels positive; significant binomial tests for two LLMs overall and for complex subsets on GPT/Gemini) supports reporting it as a real but fragile effect. The sensitivity analysis (1,350 parameter files, cross-LLM convergence, characteristic correlations) provides high-confidence evidence that adaptations reflect objective tunnel properties. Remaining uncertainty: (a) SAM and LLM **rerun** variance (not captured by tunnel bootstrap), (b) cumulative ablation cannot isolate component interactions, (c) single pipeline and dataset.

5.6 Future work

The demonstrated limitations motivate three directions:

1. **Multi-reference anchoring:** Incorporating multiple expert-tuned reference configurations (e.g., one per tunnel family) to provide stronger anchoring for tunnels that deviate substantially from a single reference.
2. **Run-to-run confidence quantification:** Repeating each condition multiple times (with different LLM seeds or SAM runs) to establish confidence intervals on reported mIoU values.
3. **Comparison with automated optimisation:** Benchmarking LLM adaptation against Bayesian optimisation or other search-based methods on the same parameter space to quantify the relative efficiency and quality of LLM-driven adaptation.

6. Conclusions

Consistent segmentation of segmental tunnel linings from point clouds remains difficult when tunnel geometry and scanning conditions vary from the expert-tuned reference. R4Tun addresses this by augmenting a fixed, expert-designed segmentation pipeline with an LLM-driven adaptation layer that adjusts stage parameters using structured context from memory, state, and domain knowledge. Evaluated on 30 Seg2Tunnel subsets across 13 regular and 17 complex tunnels with three independent LLMs, the full design improved mean mIoU from 0.150 to 0.313–0.328 (108.7–118.7% relative gain; tunnel-bootstrap 95% CIs exclude zero for all three LLMs), with state contributing the largest incremental improvement (paired Cohen’s $d > 1.3$) and all three LLMs converging on the same critical parameters and tunnel-family adaptation patterns. Confidence in the headline improvement and the dominance of state is high; confidence in the knowledge increment is low-to-moderate (bootstrap CIs straddle zero, but a majority of tunnels show positive gain); these ratings and their caveats are detailed in Table 9 (Section 5.5). For automated tunnel inspection, the framework reduces the need for expert retuning when deploying across diverse tunnel conditions while preserving full transparency through logged reasoning traces and overridable parameters. These findings should be interpreted with the caveat that absolute performance on complex tunnels remains modest (mIoU 0.169–0.177), single-run estimates carry approximately ± 0.03 noise, and transferability beyond SAM4Tun and the Seg2Tunnel dataset is untested.

Data availability

The data that support the findings of this study are available at <https://github.com/Tao-Robominds/R4Tun>. The Seg2Tunnel dataset is publicly available.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

[This work was supported by [funder] [grant number].]

Declaration of generative AI use

During the preparation of this work, the authors used large language models (Claude Opus 4.6, GPT-5.4, Gemini 3 Flash) as the core experimental subjects of the study. The LLMs were also used for language editing and organisation of the manuscript. The authors reviewed and edited all content and take full responsibility for the content of the publication.

CRedit authorship contribution statement

[Author 1: Conceptualization, Methodology, Software, Writing – original draft.] [Author 2: Supervision, Writing – review & editing.] [Author 3: Supervision, Writing – review & editing.] [Author 4: Supervision, Funding acquisition, Project administration.]

Acknowledgements

[Acknowledge non-author support.]

References

- [1] L. Attard, C. J. Debono, G. Valentino, and M. Di Castro. Tunnel inspection using photogrammetric techniques and image processing: A review. *ISPRS Journal of Photogrammetry and Remote Sensing*, 144:180–188, 2018.
- [2] L. Weidner and G. Walton. Generalized extraction of bolts, mesh, and rock in tunnel point clouds: a critical comparison of geometric feature-based methods using random forest and neural networks. *Remote Sensing*, 16(4):678, 2024.

- [3] M. Q. Huang, J. Ninić, and Q. B. Zhang. BIM, machine learning and computer vision techniques in underground construction: current status and future perspectives. *Tunnelling and Underground Space Technology*, 108:103677, 2021.
- [4] A. Sjölander, V. Belloni, A. Ansell, and E. Nordström. Towards automated inspections of tunnels: a review of optical inspections and autonomous assessment of concrete tunnel linings. *Sensors*, 23(12):5457, 2023.
- [5] R. Montero, J. G. Victores, S. Martínez, A. Jardón, and C. Balaguer. Past, present and future of robotic tunnel inspection. *Automation in Construction*, 59:99–112, 2015.
- [6] A. Strauss et al. Sensing and monitoring in tunnels: testing and monitoring methods for the assessment of tunnels. *Structural Concrete*, 21(4):1234–1248, 2020.
- [7] E. Camuffo, D. Mari, and S. Milani. Recent advancements in learning algorithms for point clouds: an updated overview. *Sensors*, 22(4):1357, 2022.
- [8] R. O. Duda and P. E. Hart. Use of the Hough transformation to detect lines and curves in pictures. *Communications of the ACM*, 15(1):11–15, 1972.
- [9] M. A. Fischler and R. C. Bolles. Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6):381–395, 1981.
- [10] M. Ester, H. P. Kriegel, J. Sander, and X. Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. *Proc. KDD*, pp. 226–231, 1996.
- [11] M. Pauly, M. Gross, and L. P. Kobbelt. Efficient simplification of point-sampled surfaces. *Proc. IEEE Visualization*, pp. 163–170, 2002.
- [12] C. R. Qi, L. Yi, H. Su, and L. J. Guibas. PointNet++: Deep hierarchical feature learning on point sets in a metric space. *NeurIPS*, 2017.
- [13] Q. Hu, B. Yang, L. Xie, S. Rosa, Y. Guo, Z. Wang, N. Trigoni, and A. Markham. RandLA-Net: Efficient semantic segmentation of large-scale point clouds. *CVPR*, 2020.
- [14] J. Schult, F. Engelmann, A. Hermans, O. Litany, S. Tang, and B. Leibe. Mask3D: Mask transformer for 3D semantic instance segmentation. *arXiv:2303.05475*, 2023.
- [15] M. Kolodiazny, A. Vorontsova, A. Konushin, and D. Rukhovich. Top-down beats bottom-up in 3D instance segmentation. *WACV*, pp. 3566–3574, 2024.
- [16] Y. J. Cha, R. Ali, J. Lewis, and O. Büyüköztürk. Deep learning-based structural health monitoring. *Automation in Construction*, 161:105324, 2024.
- [17] C. Su, Q. Hu, Z. Yang, and R. Huo. A review of deep learning applications in tunneling and underground engineering in China. *Applied Sciences*, 14(8):3234, 2024.
- [18] A. Kirillov et al. Segment Anything. *ICCV*, pp. 3992–4003, 2023.
- [19] R. Bommasani et al. On the opportunities and risks of foundation models. *arXiv:2108.07258*, 2021.
- [20] R. R. S. S. N. V. Kumar, R. S., and P. B. V. Crack-SAM: Crack segmentation using a foundation model. *arXiv:2401.15201*, 2024.

- [21] B. Wang et al. Omni-Scan2BIM: A ready-to-use Scan2BIM approach based on vision foundation models for MEP scenes. *Automation in Construction*, 167:105678, 2024.
- [22] F. Pan, S. Jeon, B. Wang, F. McKenna, and S. X. Yu. Zero-shot building attribute extraction from large-scale vision and language models. *arXiv:2312.12479*, 2023.
- [23] Z. Ye, W. Lin, A. Faramarzi, X. Xie, and J. Ninić. SAM4Tun: No-training model for tunnel lining point cloud component segmentation. *Tunnelling and Underground Space Technology*, 158:106401, 2025.
- [24] J. Bradley, A. Bran, T. Sellam, et al. ChemCrow: Augmenting large-language models with chemistry tools. *arXiv:2304.05376*, 2023.
- [25] A. Ghafarollahi and M. J. Buehler. ProtAgents: Protein discovery via large language model multi-agent collaborations combining physics and machine learning. *Digital Discovery*, 3:1956–1973, 2024.
- [26] C. Qian et al. ChatDev: Communicative agents for software development. *arXiv:2307.07924*, 2024.
- [27] S. Hong et al. MetaGPT: Meta programming for a multi-agent collaborative framework. *arXiv:2308.00352*, 2024.
- [28] P. Chen, B. Han, and S. Zhang. COMM: Collaborative multi-agent, multi-reasoning-path prompting for complex problem solving. *arXiv:2405.00847*, 2024.
- [29] C. I. Garcia et al. Framework for LLM applications in manufacturing. *Manufacturing Letters*, 40:56–63, 2024.
- [30] J. Wei et al. Chain-of-thought prompting elicits reasoning in large language models. *arXiv:2201.11903*, 2023.
- [31] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa. Large language models are zero-shot reasoners. *arXiv:2205.11916*, 2023.
- [32] V. Xiang et al. Towards System 2 reasoning in LLMs: Learning how to think with meta chain-of-thought. *arXiv:2501.04682*, 2025.
- [33] N. Shinn et al. Reflexion: Language agents with verbal reinforcement learning. *arXiv:2303.11366*, 2023.
- [34] S. Yao et al. ReAct: Synergizing reasoning and acting in language models. *arXiv:2210.03629*, 2023.
- [35] Y. Zhang et al. Chain of agents: Large language models collaborating on long-context tasks. *arXiv:2406.02818*, 2024.
- [36] Z. Zhang et al. Igniting language intelligence: the hitchhiker’s guide from chain-of-thought reasoning to language agents. *arXiv:2311.11797*, 2023.
- [37] B. Liu et al. Advances and challenges in foundation agents. *arXiv:2501.03428*, 2025.
- [38] H.-A. Gao et al. A survey of self-evolving agents: on path to artificial super intelligence. *arXiv:2501.02718*, 2025.
- [39] P. Xu et al. Retrieval meets long context large language models. *arXiv:2310.03025*, 2024.
- [40] L. Mei et al. A survey of context engineering for large language models. *arXiv:2501.04567*, 2025.
- [41] Anthropic. Effective context engineering for AI agents. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>, 2025.

- [42] OpenAI. Reasoning best practices. <https://platform.openai.com/docs/guides/reasoning-best-practices>, 2025.
- [43] M. Caron et al. Emerging properties in self-supervised vision transformers. arXiv:2104.14294, 2021.
- [44] G. Franceschelli, C. Cevenini, and M. Musolesi. Training foundation models as data compression. arXiv:2501.04023, 2025.
- [45] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell. On the dangers of stochastic parrots. Proc. FAccT, pp. 610–623, 2021.
- [46] L. Ouyang et al. Training language models to follow instructions with human feedback. arXiv:2203.02155, 2022.
- [47] S. Farquhar, J. Kossen, L. Kuhn, and Y. Gal. Detecting hallucinations in large language models using semantic entropy. Nature, 630:625–630, 2024.
- [48] T. P. Ferraz et al. LLM self-correction with DeCRIM. arXiv:2410.02919, 2024.

Appendix A. Additional experiments and reproducibility

A.1 Completed (script-only, no new pipeline runs)

Per-class IoU aggregation. Script: `methods/papers/scripts/extract_per_class_iou.py`. Outputs: [methods/papers/output/per_class_iou_summary.md](#), [methods/papers/output/per_class_iou_long.csv](#). Parses only the `## Per-class IoU` section of each `evaluation/performance.md` under `data/ablation_gpt`, `data/ablation_anthropic`, and `data/ablation_gemini`. Gemini has no `sam4tun/ tree`; **sam4tun** per-class values for Gemini rows use the shared GPT snapshot (aligned with the comparison journals).

Wilcoxon signed-rank tests. Script: `methods/papers/scripts/wilcoxon_test.py`. Output: [methods/papers/output/wilcoxon_vs_ttest.md](#). Recomputes two-sided paired t-tests and Wilcoxon tests from the per-tunnel tables in `methods/journals/comparison_openai.md`, `comparison_anthropic.md`, and `comparison_gemini.md`.

Bootstrap CIs, Cohen’s d, sign tests, spread. Script: `methods/papers/scripts/confidence_analysis.py`. Output: [methods/papers/output/confidence_analysis.md](#). Tunnel-level bootstrap (10 000 resamples) on mean paired Δ mIoU, paired Cohen’s d, one-sided binomial tests on the knowledge increment, and Table 4a / within-family standard deviations (mean across the three LLMs).

A.2 Deferred (high compute or API cost)

Run-to-run variance. Re-run the full pipeline multiple times per tunnel with fixed parameters to quantify SAM / GPU non-determinism and attach confidence intervals to headline mIoU. Planned scale: all 30 tunnels $\times$ multiple repeats $\times$ (sam4tun + m_s_k); see [methods/papers/defending_plan.md](#).

LLM stochasticity. Re-run parameter inference multiple times per LLM per tunnel under m_s_k and measure spread in mIoU. Planned scale: all 30 tunnels $\times$ 3 LLMs $\times$ multiple repeats.

Bayesian optimisation and hybrid parameter ablations are reserved for follow-on work (separate from this submission).

Appendix B. Baseline Parameter Tables

Table B.1: Unfolding parameter settings (sam4tun baseline)

Parameter	Value
delta	0.005
slice_spacing_factor	1.2
vertical_filter_window	4.5
ransac_threshold	1
ransac_probability	0.9
ransac_inlier_ratio	0.75
ransac_sample_size	5
polynomial_degree	3
num_samples_factor	1210
diameter	5.5

Table B.2: Denoising parameter settings (sam4tun baseline)

Parameter	Value
mask_r_low	2.7
mask_r_high	2.8
y_step	0.5
z_step	0.001
grad_threshold	0.2
smoothing_window_size	3
smoothing_offset	-0.003
default_cutoff_z	2.7

Table B.3: Enhancing parameter settings (sam4tun baseline)

Parameter	Value
upsampling_stage1_target_distance	0.08
upsampling_stage2_target_distance	0.04
upsampling_stage3_target_distance	0.02
curvature_threshold	0.0005
depth_threshold_low	0.003
depth_threshold_high	0.01
inter_radius	0.06
duplicate_threshold	0.02
num_neighbors	20
num_interpolations	2
resolution	0.005
window_size	9

Table B.4: Detecting parameter settings (sam4tun baseline)

Parameter	Value
binary_threshold	127
morphological_kernel_size	[3, 3]
dilation_iterations	1
hough_threshold_oblique	50
minLineLength_oblique	100
maxLineGap_oblique	40
hough_threshold_horizontal	50
minLineLength_horizontal	100
maxLineGap_horizontal	10
hough_threshold_vertical	500
angle_range_oblique_positive	[6, 9]
angle_range_oblique_negative	[-9, -6]
merge_distance	3
ring_spacing_constant	1.2
resolution	0.005